FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Calingasan, Dwayne Giusseppe P. Basister, Kian Shane
Section:	AN41
Professor:	Ms. Tan
Date:	10/03/2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA4)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base

2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

Comparative Reflection:

RBR was easier to implement because it uses simple IF statements to check conditions and choose actions. CBR required more steps, such as comparing cases, choosing the most similar case, revising the solution, and saving the new case. CBR is more flexible for less structured problems because it can use

past experiences to solve new problems. It can also handle changing situations by adding new cases, although old cases may need to be reviewed. RBR is still useful when the rules stay the same and only the input changes. Examples of RBR include appliance control and troubleshooting, while CBR can be used for help-desk support and equipment diagnosis.

Your Code - Laboratory Tasks ⇩

```
import argparse
from copy import deepcopy


# PART 1: Six facts grouped into three explicit categories.
FACTS = {
    "climate": {"temperature": "hot", "humidity": "high"},
    "weather": {"is_raining": True, "is_daylight": False},
    "room": {"is_occupied": True, "windows_open": True},
}

# Three illustrative past situations, each with features and a solution.
CASE_BASE = [
    {
        "id": "C1",
        "problem": {
            "climate": {"temperature": "hot", "humidity": "high"},
            "weather": {"is_raining": False, "is_daylight": True},
            "room": {"is_occupied": True, "windows_open": False},
        },
        "solution": "Turn on the aircon.",
    },
    {
        "id": "C2",
        "problem": {
            "climate": {"temperature": "warm", "humidity": "high"},
            "weather": {"is_raining": True, "is_daylight": False},
            "room": {"is_occupied": True, "windows_open": True},
```

```
        },
        "solution": "Close the windows. Turn on the lights.",
    },
    {
        "id": "C3",
        "problem": {
            "climate": {"temperature": "cool", "humidity": "normal"},
            "weather": {"is_raining": False, "is_daylight": True},
            "room": {"is_occupied": False, "windows_open": False},
        },
        "solution": "Keep the aircon and lights off.",
    },
]


# PART 2: Each function is one explicit IF-THEN rule.
def rule_hot(facts):
                if    facts["climate"]["temperature"]    ==    "hot"    and
facts["room"]["is_occupied"]:
        return "Turn on the aircon."


def rule_rain(facts):
    if facts["weather"]["is_raining"] and facts["room"]["windows_open"]:
        return "Close the windows."


def rule_dark(facts):
                    if     not     facts["weather"]["is_daylight"]     and
facts["room"]["is_occupied"]:
        return "Turn on the lights."


RULES = (rule_hot, rule_rain, rule_dark)


def infer(facts):
```

```
    """Check EVERY rule against working memory; return all actions."""
    actions = []
    for rule in RULES:
        action = rule(facts)
        print(f"  {rule.__name__}: {'MATCH' if action else 'NO MATCH'}")
        if action is not None:
            actions.append(action)
    return actions


# PART 3: Retrieve, reuse, revise, and retain.
def flatten(facts):
    """Use category/feature pairs to preserve the knowledge structure."""
    return {(category, key): value
            for category, values in facts.items()
            for key, value in values.items()}


def similarity(problem, past_problem):
    """Equal-weight exact matches / number of features in the query."""
    query, past = flatten(problem), flatten(past_problem)
    if not query:
            raise ValueError("The new problem must contain at least one
fact.")
    matches = sum(key in past and value == past[key]
                  for key, value in query.items())
    return matches / len(query)


def retrieve(problem, case_base):
    """Return the best case; the earliest case wins a similarity tie."""
    if not case_base:
        raise ValueError("The case base must not be empty.")
    scored = [(case, similarity(problem, case["problem"]))
              for case in case_base]
    for case, score in scored:
        print(f"  {case['id']}: similarity = {score:.2%}")
```

```
    return max(scored, key=lambda item: item[1])


def reuse(case):
    """Start with the retrieved case's existing solution."""
    return case["solution"]


def revise(solution, replacement):
    """Keep the solution when the user submits an empty response."""
    return replacement.strip() or solution


def retain(case_base, problem, solution):
    """Store an independent copy for reuse during this program session."""
    case = {"id": f"C{len(case_base) + 1}",
            "problem": deepcopy(problem), "solution": solution}
    case_base.append(case)
    return case


def main(demo=False):
    # Working memory contains the current classroom observations.
    facts, case_base = deepcopy(FACTS), deepcopy(CASE_BASE)
    print("TA4 | CLASSROOM COMFORT ADVISER")
    print("INPUT: current classroom facts")
    for category, values in facts.items():
        print(f"  {category}: {values}")

    print("\nRBR PROCESS: check all three rules")
    actions = infer(facts)
    print("Rule-Based Reasoning Actions:")
    for action in actions:
        print(f"- {action}")
    if not actions:
        print("- No actions inferred.")
```

```
    print("\nCBR PROCESS: RETRIEVE")
    case, score = retrieve(facts, case_base)
    print(f"Selected case: {case['id']} ({score:.2%})")
    solution = reuse(case)
    print(f"REUSE: {solution}")
    if demo:
        # Scripted manual revision to make the report reproducible.
        replacement = solution + " Turn on the aircon."
        print(f"REVISE (demo input): {replacement}")
    else:
        replacement = input("REVISE: new solution, or Enter to keep: ")
    solution = revise(solution, replacement)
    print(f"Final CBR solution: {solution}")
    new_case = retain(case_base, facts, solution)
            print(f"RETAIN:   added   {new_case['id']};   case   count   =
{len(case_base)}")
    print("Retention is in memory for this run; no file is written.")
    print("\nVerification: retrieve the same problem after retention")
    best, score = retrieve(facts, case_base)
    print(f"Selected case: {best['id']} ({score:.2%})")


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--demo", action="store_true",
                        help="Use the report's scripted revision.")
    main(parser.parse_args().demo)
```

Screenshots:

```
TA4 | CLASSROOM COMFORT ADVISER
INPUT: current classroom facts
  climate: {'temperature': 'hot', 'humidity': 'high'}
  weather: {'is_raining': True, 'is_daylight': False}
  room: {'is_occupied': True, 'windows_open': True}

RBR PROCESS: check all three rules
  rule_hot: MATCH
  rule_rain: MATCH
  rule_dark: MATCH
Rule-Based Reasoning Actions:
- Turn on the aircon.
- Close the windows.
- Turn on the lights.

CBR PROCESS: RETRIEVE
  C1: similarity = 50.00%
  C2: similarity = 83.33%
  C3: similarity = 0.00%
Selected case: C2 (83.33%)
REUSE: Close the windows. Turn on the lights.
REVISE: new solution, or Enter to keep:
Final CBR solution: Close the windows. Turn on the lights.
RETAIN: added C4; case count = 4
Retention is in memory for this run; no file is written.

Verification: retrieve the same problem after retention
  C1: similarity = 50.00%
  C2: similarity = 83.33%
  C3: similarity = 0.00%
  C4: similarity = 100.00%
Selected case: C4 (100.00%)
```

```
TA4 | CLASSROOM COMFORT ADVISER
INPUT: current classroom facts
  climate: {'temperature': 'hot', 'humidity': 'high'}
  weather: {'is_raining': True, 'is_daylight': False}
  room: {'is_occupied': True, 'windows_open': True}

RBR PROCESS: check all three rules
  rule_hot: MATCH
  rule_rain: MATCH
  rule_dark: MATCH
Rule-Based Reasoning Actions:
- Turn on the aircon.
- Close the windows.
- Turn on the lights.

CBR PROCESS: RETRIEVE
  C1: similarity = 50.00%
  C2: similarity = 83.33%
  C3: similarity = 0.00%
Selected case: C2 (83.33%)
REUSE: Close the windows. Turn on the lights.
REVISE: new solution, or Enter to keep:
```

```
TA4 | CLASSROOM COMFORT ADVISER
INPUT: current classroom facts
  climate: {'temperature': 'hot', 'humidity': 'high'}
  weather: {'is_raining': True, 'is_daylight': False}
  room: {'is_occupied': True, 'windows_open': True}

RBR PROCESS: check all three rules
  rule_hot: MATCH
  rule_rain: MATCH
  rule_dark: MATCH
Rule-Based Reasoning Actions:
- Turn on the aircon.
- Close the windows.
- Turn on the lights.

CBR PROCESS: RETRIEVE
  C1: similarity = 50.00%
  C2: similarity = 83.33%
  C3: similarity = 0.00%
Selected case: C2 (83.33%)
REUSE: Close the windows. Turn on the lights.
REVISE: new solution, or Enter to keep: Close the windows. Turn on the lights. Turn on the aircon.
Final CBR solution: Close the windows. Turn on the lights. Turn on the aircon.
RETAIN: added C4; case count = 4
Retention is in memory for this run; no file is written.

Verification: retrieve the same problem after retention
  C1: similarity = 50.00%
  C2: similarity = 83.33%
  C3: similarity = 0.00%
  C4: similarity = 100.00%
Selected case: C4 (100.00%)
```